

NAME: K.Naga Venkata Siva

REGNO: 192425345

GEMINI:

Question 1: Smartphone Performance Optimization

A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

- **Analysis of Memory Layers:**
 - **L1 Cache (Core-Private):** Split into instruction ($L1\text{-I}$) and data ($L1\text{-D}$) caches, operating at CPU core clock speeds ($<1\text{ ns}$) with typical sizes of $32\text{--}64\text{ KB}$. It holds the immediate operational state and active execution loops.
 - **L2 Cache (Cluster-Private):** Slightly larger ($512\text{ KB--}1\text{ MB}$) with $3\text{--}5\text{ ns}$ access times; acts as an intermediate buffer to catch inner-loop L1 evictions.
 - **L3 Cache / System-Level Cache (Shared):** Shared across CPU, GPU, and NPU ($4\text{--}16\text{ MB}$; $10\text{--}20\text{ ns}$ latency); retains working sets across background applications.
 - **DRAM (LPDDR5/5X):** High-capacity main system memory with $100\text{--}200\text{ ns}$ latency, delivering $50\text{--}60\text{ GB/s}$ bandwidth.
- **Root Cause:** App switching forces a CPU context switch. The incoming application suffers cold misses across L1/L2 caches, stalling execution while data is refetched from high-latency LPDDR DRAM.
- **Redesign Solution:**
 - Implement an on-chip **System-Level Cache (SLC)** outside the CPU cluster to buffer recently switched background application states before writing back to DRAM.
 - Employ **victim caches** and dynamic cache-partitioning algorithms to reserve cache ways for foreground UI-thread critical paths.
 - Integrate predictive hardware prefetchers coupled with OS scheduler hints to preload the incoming app context into L2/L3 before the context switch finishes.

Question 2: Cloud Server Memory Bottleneck

A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

- **Cache Hierarchy vs. Virtual Memory Paging:**
 - **Cache Hierarchy:** Entirely hardware-managed (SRAM); operates on fixed cache lines (usually 64 bytes) with latencies ranging from 1 ns to 20 ns. Cache misses trigger memory bus access cycles.
 - **Virtual Memory Paging:** Managed cooperatively by the OS and Memory Management Unit (MMU); handles data in pages (4 KB, 2 MB, or 1 GB) using Translation Lookaside Buffers (TLB) and page tables. Page faults incur heavy latency penalties (microseconds to milliseconds) from disk or network storage fetches.
- **Root Cause:** Large database workloads with non-contiguous indexing and pointer chasing cause frequent TLB evictions, high page table walk overhead, and memory bus congestion.
- **Recommended Improvements:**
 - **Transparent Huge Pages (2 MB / 1 GB):** Reduce the depth of page table walks and expand effective TLB coverage.
 - **Compute Express Link (CXL):** Deploy CXL-attached memory pooling over PCIe Gen 5/6 to expand byte-addressable DRAM pools with minimal latency overhead.
 - **Persistent Memory / Storage-Class Memory (SCM):** Place active database indexes and write-ahead logs in byte-addressable non-volatile memory to avoid disk-bound serialization.

Question 3: Autonomous Vehicle Data Processing

An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

- **Evaluation of Memory Technologies:**
 - **SRAM:** Volatile, 6T configuration, ultra-fast access (< 1 ns), high bandwidth; drawback is large physical footprint and high static leakage power.
 - **DRAM:** Volatile, 1T-1C cell, high density and cost-effective; drawback is higher latency (50 ns to 80 ns) and periodic refresh cycles that introduce nondeterministic latency spikes.
 - **MRAM (Magnetoresistive RAM):** Non-volatile (spin-torque transfer / MTJ), zero standby leakage, near-SRAM read speeds, high endurance, radiation-tolerant, and provides instant data retention upon power loss.
- **Proposed Memory Hierarchy Design:**

- **Level 0/1 Scratchpad (SRAM):** Embedded directly inside vision accelerators and sensor-fusion DSPs for cycle-accurate, zero-latency processing of raw LiDAR and radar frames.
- **Level 2 Non-Volatile Mission-Critical Storage (MRAM):** Used for real-time localization tables, vehicle trajectory states, and instant crash/blackbox logging without requiring backup capacitors.
- **Level 3 High-Capacity Buffer (LPDDR5 / HBM):** High-bandwidth shared memory to store large neural network perception models and deep buffer video streams.

Question 4: AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

- **Comparison of Memory Technologies:**
 - **NAND Flash:** High storage density and non-volatile; however, block-level reads/writes, slow random access (μs range), and high active energy make direct execution inefficient.
 - **DRAM:** High data transfer rates; however, continuous capacitor refresh requirements cause significant idle/standby power leakage ($>50\text{ mW}$).
 - **RRAM (Resistive RAM):** Non-volatile memristor technology, byte-addressable, high read speeds, low switching energy, and native support for analog In-Memory Computing.
- **Optimized Hierarchical Memory Structure:**
 - **Non-Volatile Model Storage (RRAM Array):** Store quantized neural network weights directly in an embedded RRAM macro, enabling instant wake-up without loading large weights over external buses.
 - **Compute-In-Memory (CiM) Core:** Perform vector-matrix multiplications directly inside the RRAM crossbar array in the analog/charge domain to eliminate data movement bottlenecks.
 - **Activation Buffer (On-chip SRAM):** Use a small SRAM buffer to store dynamic layer activations and intermediate feature maps between layers.

Question 5: High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

- **L3 Cache vs. DRAM Throughput Analysis:**

- **L3 Cache:** Delivers massive local bandwidth ($1\text{--}2\text{ TB/s}$) at low latency ($15\text{--}25\text{ ns}$), but limited capacity ($16\text{--}32\text{ MB}$) cannot hold modern 4K/8K scene textures and geometry datasets.
- **DRAM (GDDR6 / HBM):** High capacity ($16\text{--}24\text{ GB}$) with $400\text{--}900\text{ GB/s}$ throughput; during scene transitions, simultaneous CPU asset requests and GPU draw calls overwhelm memory channels, causing bus contention and frame stutter.
- **Memory Hierarchy Enhancements:**
 - **Unified Memory Architecture (UMA):** Share a single, high-bandwidth GDDR6/HBM memory pool between CPU and GPU with zero-copy addressing, eliminating CPU-to-GPU data duplication.
 - **Hardware Sampler Feedback & Direct SSD-to-Memory Streaming:** Stream only visible mipmap tiles from an NVMe SSD through a dedicated hardware decompression engine directly into DRAM/L3, bypassing CPU decompression overhead.
 - **Large On-Die Intermediate Cache (Infinity Cache / eDRAM):** Implement a $64\text{--}128\text{ MB}$ dedicated high-speed SRAM cache layer to absorb frame-buffer operations (MSAA, ray tracing bounding boxes, render targets) and free up DRAM bandwidth for asset streaming.

CHATGPT:

Question 1: Smartphone Performance Optimization

A smartphone manufacturer notices that applications take time to respond when users switch between apps. This problem can be reduced by improving the memory hierarchy and minimizing slow DRAM accesses.

Role of Memory Layers

- **L1 Cache:**
The fastest and smallest cache located inside each CPU core. It stores frequently used instructions and data for immediate processing.
- **L2 Cache:**
Larger than L1 and slightly slower. It stores recently used information that is removed from L1, reducing the need to access main memory.
- **L3 Cache / System-Level Cache:**
A larger shared cache that can serve CPU, GPU, and other processing units. It helps maintain frequently used application data during multitasking.

- **DRAM:**

Provides large-capacity main memory for applications and the operating system. However, it has considerably higher latency than cache memory.

Main Problem

During application switching, the new application may not have its required data in the CPU caches. This creates **cache misses**, causing the processor to retrieve information from DRAM. Repeated DRAM accesses increase latency and create noticeable lag.

Proposed Redesign

- Increase the capacity of the **System-Level Cache (SLC)**.
- Use **victim caches** to retain recently evicted data.
- Dynamically allocate cache space to important foreground applications.
- Use **hardware prefetching** to predict and load required data early.
- Provide cache and scheduling hints through the operating system.
- Keep frequently accessed application data closer to the processor.

Expected Result

Fewer cache misses → fewer DRAM accesses → lower latency → faster app switching → better user experience.

Question 2: Cloud Server Memory Bottleneck

A cloud service provider experiences slow database queries because large databases continuously access memory. Efficient use of cache and virtual memory can reduce these delays.

Cache Hierarchy

- Cache memory uses fast **SRAM** and is managed mainly by hardware.
- It stores frequently accessed data in small cache lines.

- L1, L2, and L3 caches provide progressively larger but slower storage.
- A cache hit allows the processor to obtain data quickly.

Virtual Memory

- Virtual memory is controlled by the **operating system and MMU**.
- Data is organized into pages such as 4 KB, 2 MB, or larger pages.
- The **TLB** stores recent virtual-to-physical address translations.
- A TLB miss can require page-table walks.
- A page fault can be extremely expensive because data may need to be retrieved from storage.

Main Problem

Large database operations involving random memory access and pointer-based data structures can cause:

- Frequent cache misses
- TLB misses
- Additional page-table walks
- Increased memory traffic
- Higher database response time

Recommended Improvements

- Use **Huge Pages** to increase TLB coverage and reduce address-translation overhead.
- Use **CXL-attached memory** to expand available memory capacity.
- Keep frequently accessed database indexes in faster memory.
- Use suitable persistent-memory technologies for important database data and logs.
- Optimize database data structures to improve locality of reference.

Expected Result

Better locality + fewer TLB misses + greater memory capacity → faster database queries and improved server performance.

Question 3: Autonomous Vehicle Data Processing

An autonomous vehicle must process LiDAR, radar, camera, and other sensor information with very low delay. A multi-level memory system is required to provide both high speed and sufficient capacity.

Comparison of Memory Technologies

- **SRAM:**
Extremely fast and provides high bandwidth. It is suitable for small amounts of data that require immediate processing, but it consumes more chip area.
- **DRAM:**
Provides high capacity at a reasonable cost and is suitable for storing large sensor datasets. However, it is slower than SRAM and requires periodic refresh operations.
- **MRAM:**
Non-volatile memory that provides fast access and retains information without continuous power. It is useful for mission-critical information and rapid recovery.

Proposed Memory Hierarchy

Level 1 – SRAM

- Place small SRAM buffers close to sensor-processing units.
- Store current LiDAR, radar, and camera data.
- Provide fast access for real-time calculations.

Level 2 – MRAM

- Store important vehicle states and navigation information.
- Maintain localization and trajectory data.
- Support reliable event and emergency logging.

Level 3 – LPDDR5 / HBM

- Store large sensor datasets.

- Hold AI models and video buffers.
- Provide high-bandwidth access for perception and sensor-fusion workloads.

Expected Result

SRAM provides speed → MRAM provides reliable persistent storage → DRAM/HBM provides capacity and bandwidth.

This hierarchy enables fast and reliable real-time vehicle processing.

Question 4: AI Inference Edge Device

An edge AI device needs to load AI models quickly while consuming as little power as possible. Combining non-volatile memory with fast on-chip memory can improve both performance and energy efficiency.

Comparison of Memory Technologies

- **NAND Flash:**
Provides high storage capacity and retains data without power. However, random access is relatively slow, making it unsuitable for frequent direct model operations.
- **DRAM:**
Provides high-speed data access and is suitable for temporary AI data. However, it requires continuous refresh, which increases power consumption.
- **RRAM:**
Non-volatile memory with fast access and low-energy operation. It can also support **Compute-in-Memory (CiM)** operations.

Proposed Memory Hierarchy

Level 1 – RRAM Model Storage

- Store frequently used neural-network weights directly in RRAM.
- Reduce the need to reload large models from external storage.

- Enable faster device startup.

Level 2 – Compute-in-Memory

- Perform suitable matrix and vector operations close to the stored weights.
- Reduce movement of data between memory and processor.
- Improve energy efficiency.

Level 3 – On-Chip SRAM

- Store temporary activations and intermediate results.
- Provide very fast access during neural-network execution.

Expected Result

RRAM stores the model → CiM reduces data movement → SRAM handles temporary data → faster and lower-power AI inference.

Question 5: High-Performance Gaming Console

A gaming console may experience frame drops when loading large environments because the CPU and GPU compete for memory bandwidth while loading textures, geometry, and other game assets.

L3 Cache vs DRAM

- **L3 Cache:**
Very fast and provides high local bandwidth, but its capacity is limited. It is suitable for frequently reused data and rendering-related operations.
- **GDDR6 / HBM:**
Provides much larger capacity and high memory bandwidth. It can store large textures, game assets, frame buffers, and geometry.

Main Problem

During large scene loading:

- CPU requests game assets.
- GPU simultaneously requests rendering data.
- Multiple memory requests compete for available bandwidth.
- Memory congestion can delay rendering operations.
- This may result in **frame drops and stuttering**.

Proposed Memory Hierarchy

Level 1 – CPU/GPU Cache

- Keep frequently reused instructions, textures, and rendering data close to the processing units.

Level 2 – Large Shared On-Chip Cache

- Add a larger cache layer to temporarily hold frequently accessed rendering data.
- Reduce unnecessary DRAM traffic.

Level 3 – High-Bandwidth GDDR6/HBM

- Store large game assets, textures, geometry, and frame buffers.
- Provide high bandwidth for GPU workloads.

Level 4 – NVMe SSD

- Store the complete game library and large asset collections.
- Stream only the required assets into memory.

Additional Optimization

- Use **Unified Memory Architecture (UMA)** to allow CPU and GPU to efficiently share memory.
- Stream only required texture and geometry data instead of loading complete assets.
- Use hardware decompression to reduce CPU workload.
- Use intelligent caching to keep frequently used game assets available.

Expected Result

**Fast cache → high-bandwidth DRAM → intelligent SSD streaming →
reduced memory contention → smoother rendering and fewer frame drops.**